CS415 SOFTWARE ENGINEERING

Final Project Report

LifeLink: A Blood Donation Matching Platform for Ghana

Submitted by: Group 3

Course Instructor: Dr. Umut Tosun

Submission Date: 1st May 2026

Table of Contents
1. Executive Summary
2. Project Overview
3. System Description
4. Technology Stack and Architecture
5. Software Engineering Methodology
6. Design Patterns Implementation
7. Database Design
8. Testing and Quality Assurance
9. Project Management
10. Conclusion and Future Work

Links

Github: https://github.com/Dounia337/lifelink.git

Figma Design: https://www.figma.com/design/DbqFZFoCY9vohOFZ1ANQU1/LifeLink?node-id=33-224&t=TRwqG6jnvTzR4haJ-1

Live Website: http://169.239.251.102:280/~deubaybe.dounia/lifelink/

1. Executive Summary

This report documents the development of LifeLink, a web-based blood donation matching platform designed to address Ghana's critical blood shortage.

LifeLink connects hospitals in need of blood with verified donors in real time through an automated matching algorithm. The platform addresses Ghana's approximately 40% blood shortage gap by providing a centralized registry of blood donors, enabling hospitals to create emergency blood requests, and automatically matching them with compatible donors based on blood type compatibility and location proximity.

The system was built using a three-tier architecture with a PHP backend, MySQL database, and a frontend built with HTML, Tailwind CSS, and Vanilla JavaScript. Three classical design patterns, **Singleton, Factory, and Observer** were implemented to demonstrate software engineering best practices. Development followed an Agile methodology with iterative sprints and continuous team feedback.

2. Project Overview

2.1 Problem Statement

Access to timely blood transfusions is critical for managing road accidents, obstetric emergencies, severe anaemia, and major surgeries in Ghana. The National Blood Service collected approximately 187,280 units of blood in 2024 against a WHO-recommended need of 308,000 units, representing a shortfall of more than 120,000 units, equivalent to approximately 40% of the national requirement.

Ghana's blood collection index rose marginally from 5.9 to 6.1 donations per 1,000 people between 2023 and 2024, still well below the WHO benchmark of 10 donations per 1,000 population. Voluntary unpaid donations also declined from 40% in 2023 to 29% in 2024, increasing reliance on last-minute family replacement donors.

2.2 Project Objectives

The platform was designed to:

- Provide a centralised and accessible registry of blood donors
- Allow donors to register even if they do not know their blood type
- Support blood type verification through health workers or blood drives
- Enable hospitals to create emergency blood requests with urgency levels

- Automatically match hospitals with nearby compatible donors
- Prioritise verified donors in matching results
- Track request status from open through to fulfilment
- Provide basic analytics on donation activities and response times

2.3 Team Composition

Name	Role
Deubaybe Dounia	Scrum Master
Inares Kenn Tsangue	Product Owner
Ibrahim Mahamadou Abdou	Full Stack Developer
Abdoul Akim N'goila Karimou	Frontend Lead

3. System Description

3.1 User Roles

User Type	Primary Responsibilities
Donor	Register profile, update availability, view requests, track donation history
Hospital / Clinic	Create blood requests, view matched donors, track request status
Health Worker	Verify donor blood types, update medical records, view verification logs
Administrator	Manage user accounts, approve institutions, monitor system analytics

3.2 Key Features

The donor module allows users to register with an optional unknown blood type, manage their profile including contact and eligibility details, track their donation history, and toggle availability. The hospital module supports the creation of blood requests specifying blood type, number of units, urgency level, and patient details, as well as the ability to view ranked matched donors and update request statuses.

The matching engine applies blood type compatibility rules, calculates location-based proximity using the Haversine formula, prioritises verified donors, and considers urgency level. A verification module enables health workers to update donor blood types and maintain a full audit log. The administrator dashboard provides system-wide statistics, user management, and hospital approval workflows.

4. Technology Stack and System Architecture

4.1 Technology Stack

Layer	Technology	Justification
Frontend	HTML5 + Tailwind CSS	Lightweight styling, no build step required
Frontend	Vanilla JavaScript	Dynamic rendering and API calls with no framework overhead
Backend	PHP	Team expertise and rapid development of REST API endpoints
Database	MySQL	Relational data store capable of handling complex joins and queries
Authentication	PHP Sessions	Stateful login and server-side role enforcement
Design Tool	Figma	UI/UX prototyping and wireframes before implementation
Development Env.	XAMPP	Consistent local development environment across all team members

4.2 Three-Tier Architecture

LifeLink is structured as a three-tier application with a clear separation between the presentation, business logic, and data layers.

The Presentation Tier runs entirely in the browser and consists of HTML pages styled with Tailwind CSS, with a shared JavaScript file (app.js) managing the API client, authentication checks, navigation rendering, toast notifications, and data formatting. No business logic is present in this layer.

The Business Logic Tier is implemented in PHP and exposed as a REST API. It is partitioned by domain responsibility: auth.php handles login, registration, and session management; requests.php manages blood request lifecycle and the matching engine; donors.php handles donor profile data and blood type verification; and admin.php manages system statistics and user operations. The three design patterns, **Database (Singleton), UserFactory (Factory)**, and **EventSystem (Observer)** operate at this tier.

The Data Tier consists of a MySQL database accessed exclusively through PDO prepared statements. All SQL operations are centralized within the PHP files; the frontend layers have no direct database access. The database schema includes nine tables: users, donor profiles, hospitals, blood requests, donor matches, notifications, donation records, verification logs, and a blood compatibility reference matrix.

4.3 API Communication

Every frontend page loads the shared app.js module which provides a centralized API client. All requests are transmitted as JSON over HTTP and include session credentials. On each request, the server validates the active session and enforces role-based access control before executing any

business logic. The API supports standard HTTP methods (GET, POST, PUT) mapped to the corresponding resource endpoints.

5. Software Engineering Methodology

5.1 Agile Development Approach

The project followed Agile methodology using a Scrum-style approach across four two-week sprints. Sprint 1 covered project setup, database schema, and authentication. Sprint 2 delivered the donor and hospital modules along with the matching engine. Sprint 3 implemented the admin dashboard, health worker module, and notification system. Sprint 4 was dedicated to testing, bug fixing, and final documentation.

The team maintained a prioritized product backlog managed in **ClickUp**, conducted daily stand-ups to communicate progress and blockers, and reviewed deliverables at the end of each sprint. A **Git branching** strategy using feature branches and pull requests was adopted for version control.

5.2 Core Engineering Principles

Four key software engineering principles were applied throughout the project. **Separation of Concerns** was enforced by dividing the system into distinct presentation, logic, and data layers. **Modularity** was applied by centralizing shared behavior in app.js on the frontend and config.php on the backend, ensuring that each component was cohesive and focused on a single, well-constrained function. **Functional Independence** was upheld by ensuring each PHP file owned exactly one domain of logic, with high cohesion and low coupling to other components. **Information Hiding** was implemented on the server at every endpoint, with frontend mirroring to redirect users who access pages outside their role, ensuring that internal data structures and processing details remained inaccessible to unauthorized modules.

6. Design Patterns Implementation

Three patterns were implemented to solve concrete problems in the system and to demonstrate software engineering best practices.

6.1 Singleton Pattern Database.php

Category: Creational. **Problem:** Establishing a database connection is resource intensive. Multiple simultaneous connections would waste resources and risk data consistency issues. **Solution:** The Database class uses a private static instance variable and a private constructor to ensure that only one PDO connection is created for the lifetime of each request. The public static getInstance() method returns the existing instance or creates it on first call. A private clone method prevents duplication. All PHP API files obtain the database connection exclusively through this pattern.

6.2 Factory Pattern UserFactory.php

Category: Creational. Problem: The registration handler originally contained a lengthy conditional block that inserted records directly into different profile tables depending on the user role. This made adding new roles error-prone. **Solution:** An abstract UserCreator class defines createBaseUser() and createProfile() methods. Concrete subclasses DonorCreator, HospitalCreator, and HealthWorkerCreator each implement createProfile() for their specific table. The UserFactory class exposes a static getCreator() method that returns the correct creator for a given role string. The registration handler delegates all creation logic to the factory, eliminating the conditional block entirely.

6.3 Observer Pattern EventSystem.php

Category: Behavioural. Problem: The platform needs to notify multiple components, in-app notifications, dashboard updates, and future SMS or email channels, when domain events occur, such as a new blood request or a successful donor match. Hardcoding these notifications inside the business logic would tightly couple unrelated concerns. **Solution:** An EventObserver interface defines an update() method. The EventSystem class maintains a registry of observers keyed by event name and exposes attach() and notify() methods. Concrete observers, such as NotificationObserver, implement the interface and persist notification records to the database. Business logic files simply call notify() with an event name and payload, remaining unaware of which observers are registered.

7. Database Design

7.1 Schema Overview

The relational schema consists of nine tables designed to support the full lifecycle of blood donation matching. The users table serves as the base entity for all roles, storing shared identity attributes. Role-specific data is held in the donor_profiles and hospitals tables, each linked to users via a one-to-one foreign key relationship.

Table	Description
users	Base table storing identity, role, contact, city, and active status for every account
donor_profiles	Extends users for donors: blood type, weight, verification status, last donation date, and availability
hospitals	Extends users for hospitals: hospital name, registration number, type, and approval status
blood_requests	Records each hospital request: blood type, units needed, urgency, current status, and timestamps
donor_matches	Junction table linking a blood request to each matched donor, tracking notification and response status

notifications	Stores in-app notification messages for each user, generated by the Observer pattern
donation_records	Permanent audit log of completed donations, linking donor, request, date, and units
verification_log	Audit trail recording each blood type verification event performed by a health worker
blood_compatibility	Reference matrix defining which donor blood types are compatible with each recipient type

7.2 Matching Algorithm

The matching engine is triggered each time a new blood request is created. It operates in five sequential steps:

- Retrieve the set of donor blood types compatible with the requested type using the blood_compatibility reference table.
- Query donor_profiles for available, active donors whose blood type falls within the compatible set.
- Sort the candidate list by verification status descending (verified donors appear first) and then by proximity ascending.
- Select the top ten candidates, insert corresponding records into donor_matches, and trigger the EventSystem to dispatch notifications to each matched donor.

7.3 Data Integrity Measures

All foreign key constraints are enforced with ON DELETE CASCADE where appropriate to prevent orphaned records. SQL injection is eliminated by using PDO prepared statements with bound parameters throughout all PHP API files. Input sanitization is applied at the API boundary before any values reach the query layer. Role and status fields use MySQL ENUM types to restrict values to defined sets.

8. Testing and Quality Assurance

8.1 Testing Strategy

Given the project's academic scope and timeline, the team adopted a manual testing strategy covering all four user roles. Tests were executed against a locally seeded database to ensure consistent and reproducible results. End-to-end workflows were verified for login, registration, request creation, donor matching, blood type verification, and administrative operations.

8.2 Key Test Cases by Role

Role	Test Case	Expected Outcome
Donor	Register with unknown blood type	Profile created with 'unknown' type, available for future verification
Donor	Toggle availability	Status updated in donor_profiles table immediately

Hospital	Create blood request	Request persisted, matching engine triggered, donors notified
Hospital	Mark request as fulfilled	Status updated, donation record created in audit log
Health Worker	Verify a donor's blood type	Donor status changed to verified, verification_log entry created
Admin	Approve a hospital account	Hospital is_approved flag set, institution gains request access
Admin	Deactivate a user	is_active flag set to false, user cannot log in

8.3 Challenges and Solutions

Challenge	Solution
Blood type compatibility complexity	Created a dedicated blood_compatibility reference table and queried it at runtime
Decoupled notifications across components	Adopted the Observer pattern (EventSystem) to avoid tight coupling
Consistent local development environment	Standardised on XAMPP and shared the schema.sql with seed data
Scope management during sprints	Deferred lower-priority features (mobile app, SMS) to the future work backlog

9. Project Management

9.1 Sprint Summary

Sprint	Weeks	Deliverables
Sprint 1	Weeks 1–2	Project setup, database schema, authentication module
Sprint 2	Weeks 3–4	Donor module, hospital module, matching engine
Sprint 3	Weeks 5–6	Admin dashboard, health worker module, notification system
Sprint 4	Weeks 7–8	Testing, bug resolution, documentation, final presentation

9.2 Tools Used

Tool	Purpose
Git + GitHub	Version control, feature branching, and collaborative code review via pull requests
ClickUp	Sprint board, task tracking, burndown charts, and backlog management
Figma	UI/UX design, wireframing, and component prototyping before implementation
VS Code	Primary integrated development environment for all team members
XAMPP	Local web server stack providing Apache, PHP, and MySQL in one package
phpMyAdmin	Database administration and schema inspection during development

10. Conclusion and Future Work

10.1 Project Achievements

LifeLink successfully demonstrates a functional blood donation matching platform that addresses a documented and pressing public health need in Ghana. The key achievements of the project are as follows:

- A working functional prototype that performs real-time blood donor matching based on compatibility
- Rigorous application of Agile methodology, including sprint planning, daily stand-ups, and retrospectives.
- Implementation of three classical design patterns that solved concrete problems in the codebase rather than being applied artificially.
- A complete four-role access control system with both server-side enforcement and front-end session management.
- A health worker verification interface that maintains a full audit trail of blood type updates.
- A system administration dashboard providing real-time statistics and user management capabilities.

10.2 Lessons Learned

Several important lessons emerged from the development process. Thorough requirements gathering and early database schema design significantly reduced rework in later sprints. The application of design patterns proved valuable beyond academic exercise: the Singleton prevented redundant database connections, the Factory simplified the addition of new user roles, and the Observer enabled clean decoupling of the notification subsystem. Manual testing, while labour-intensive, was effective in surfacing integration issues that would have been costly to resolve in a production environment.

10.3 Future Enhancements

Feature	Description	Priority
Mobile Application	Cross Platform applications for broader access in low-connectivity areas	High
SMS Notifications	Text message alerts for emergency requests targeting donors without smartphones	High
Blood Drive Events	A module for scheduling and managing community donation events	Medium
Advanced Analytics	Enhanced reporting dashboards with trend visualization for administrators	Medium
Donor Rewards	Gamification system with badges and points to incentivize repeat donations	Low

Multi-language Support	Localizations for Ghana's principal local languages	Low

References

Head, M., Shen, H., & Brede, M. (2026, January 15). Ghana collects half the blood it needs – digital approaches can improve that. The Conversation. https://theconversation.com/ghana-collects-halfthe-blood-it-needs-digital-approaches-can-improve-that-271436

Graphic Online. (2025, January). Ghana News: Blood Service urges support to meet 308,000 units annually. Graphic Online. https://www.graphic.com.gh/news/general-news/ghana-news-bloodservice-urges-support-to-meet-308-000-units-annually.html

World Health Organization Regional Oflice for Africa. (2023, June 14). Low donor numbers hamper Africa's national blood services. WHO Regional Oflice for Africa. https://www.afro.who.int/news/low-donor-numbers-hamper-africas-national-blood-services

16_EBOOK-
7th_ed_software_engineering_a_practitioners_approach_by_roger_s._pressman_.pdf